

Documentación Evolutiva del Proyecto: Body Paint

Grupo 6

I. Integrantes

<i>Nombre y Apellido</i>	<i>Mail</i>	<i>Legajo</i>	<i>Rol Interno (Opcional)</i>
Lautaro Oliviero	lautarooliviero44@gmail.com	17428	Scrum Master / Developer
Mauricio Encina	maurixmre@gmail.com	17360	Developer
Luciana Sassia	sassialuciana@gmail.com	17475	Developer

II. Enlaces del Proyecto

Repositorio en GitHub:

<https://github.com/LautiOliviero/TRABAJO-PR-CTICO-INTEGRADOR-TPI-MSII---GRUPO-6.git>

Proyecto en Jira:

<https://grupo6msll.atlassian.net/jira/software/projects/SCRUM/boards/1?atlOrigin=eyJpIjoiMjllODRlZGExZjVjNGUxODhIMjcwMTU5N2YwMjI2MDgiLCJwIjoiajI9>

III. Roles del Sistema y Responsabilidades

- **Cliente:** Usuario que navega el catálogo, arma carros de compras y confirma pedidos con pago contra entrega.
- **Vendedor:** Usuario interno de Body Paint que visualiza y gestiona los estados de los pedidos de los clientes.
- **Administrador de Productos:** Usuario interno encargado de mantener actualizado el catálogo de productos, gestionar el stock, cargar nuevos productos y visualizar las alertas de inventario.

(Nota: El rol del Product Owner (PO) es ejercido por los docentes de la cátedra).

IV. Registro de Sprints

SPRINT 0

I. Objetivo del Sprint:

Realizar el análisis funcional y técnico del dominio 'Body Paint', configurar las herramientas de gestión (Jira y GitHub) y definir los criterios de aceptación y estimaciones para las User Stories del cliente y el MVP.

II. User Stories seleccionadas para el Sprint Backlog:

US-01: Cupón de Descuento

US-02: Gestión de Reclamos

US-03: Solicitud de Reenvío

III. Detalle de cada User Story:

US-01: Cupón de Descuento

- **Descripción:** Como cliente, quiero ingresar un cupón de descuento para obtener beneficios en mi compra.
- **Criterios de Aceptación:**

1. El sistema debe permitir el ingreso de un código de cupón en el carro de compras.

2. El descuento solo se aplica si el monto del pedido es mayor al descuento obtenido.

3. Al confirmar el pago, el cupón debe marcarse como usado y quedar asociado al pedido.

- **Estimación:** Requiere validación de reglas de negocio y actualización de montos en tiempo real.

- o Esfuerzo: 5 (Escala Fibonacci).

- o Complejidad: Media.

- o Incertidumbre: Baja.

US-02: Gestión de Reclamos

- **Descripción:** Como cliente, quiero realizar reclamos dentro de los 5 días posteriores a la entrega para solucionar problemas.

- **Criterios de Aceptación:**

- 1. El sistema debe habilitar la opción de reclamo solo hasta 5 días hábiles después de la entrega.

- 2. Si no hay reclamo en ese plazo, el pedido se marca como finalizado automáticamente.

- **Estimación:** Esfuerzo bajo ya que se reutilizará la lógica de validación de fechas de proyectos anteriores, aunque con incertidumbre media por la integración de feriados.

- o Esfuerzo: 3.

- o Complejidad: Baja.

- o Incertidumbre: Media (por el cálculo de días hábiles).

US-03: Solicitud de Reenvío

- **Descripción:** Como cliente, quiero solicitar el reenvío de productos para recibir lo faltante o defectuoso.

- **Criterios de Aceptación:**

- 1. El cliente puede elegir si el reenvío es por la totalidad o una parte del pedido.

2. La solicitud debe quedar sujeta a un análisis previo del vendedor.

- **Estimación:** Se considera de esfuerzo alto no solo por la lógica de selección de productos, sino por la incertidumbre crítica respecto a la disponibilidad de stock. Si el producto a reenviar no está disponible, el sistema debe contemplar un flujo alternativo (como nota de crédito o espera de reposición), lo que aumenta el riesgo técnico y de negocio.

- o Esfuerzo: 8.

- o Complejidad: Alta (implica lógica de logística y nuevos estados de pedido).

- o Incertidumbre: Alta.

IV. Resultado del Sprint:

Funcionalidades implementadas:

- Configuración del entorno de gestión: Setup de Jira Software con tablero Scrum y creación del repositorio organizacional en GitHub.
- Análisis funcional y técnico: Definición de Criterios de Aceptación y estimaciones Fibonacci para las historias de usuario asignadas.
- Definición de Stack Tecnológico: Selección de HTML/JS/Bootstrap y MockAPI como arquitectura base.

Funcionalidades pendientes o replanificadas:

El desarrollo del código fuente de las User Stories (US-01, US-02, US-03) se traslada al Sprint 3, priorizando en esta etapa la base estructural y técnica del MVP.

V. Decisiones relevantes tomadas por el equipo:

Decisiones técnicas:

1. Actualización de Stock en tiempo real (Persistencia): Se optó por una arquitectura basada en servicios externos mediante MockAPI, que permite la actualización automática de las cantidades de productos cada vez que se confirme un pedido. Esto garantiza que el Administrador visualice datos reales y actualizados, cumpliendo con el requisito de visibilidad del stock de forma inmediata.

2. Tecnología: Se utilizará un stack basado en HTML, CSS (Bootstrap) y JavaScript. Esta decisión se fundamenta en la experiencia previa del equipo con estas herramientas, lo que garantiza agilidad en el desarrollo del Frontend y asegura la entrega del MVP en los plazos establecidos, minimizando el riesgo técnico.

SPRINT 1

VI. Objetivo del Sprint:

Configurar el entorno de desarrollo local, establecer la arquitectura de carpetas del proyecto y diseñar la integración inicial con la API externa (MockAPI) para renderizar el catálogo de productos de forma dinámica sobre una interfaz responsiva base.

VII. Épicas y Tareas seleccionadas para el Sprint Backlog:

Épica: Desarrollo de la Base Estructural - Parte I: Infraestructura y Datos

Tarea 1: Configuración de estructura de directorios (/css, /js, /assets).

Tarea 2: Creación de `index.html` base con componentes comunes.

Tarea 3: Setup de recursos y datos iniciales en MockAPI.

Tarea 4: Desarrollo de la lógica de Fetch para carga de catálogo.

Tarea 5: Maquetado de Cards de productos con Bootstrap.

VIII. Detalle de la Épica y sus Tareas Internas:

Épica: Desarrollo de la Base Estructural - Parte I: Infraestructura y Datos

- **Descripción:** Configuración inicial del entorno de desarrollo, arquitectura de carpetas y establecimiento de la comunicación con la API externa. El objetivo es tener el catálogo de productos funcional y el esqueleto visual del sitio listo para recibir la lógica de negocio.

- **Criterios de Aceptación:**

1. Estructura Base: El proyecto debe contar con la arquitectura de directorios definida y una plantilla base (Boilerplate) con Navbar y Footer funcionales.
2. Conexión a MockAPI: Se deben haber creado las colecciones de clientes, productos y pedidos en la plataforma.
3. Catálogo Dinámico: Los productos mostrados en la galería deben consumirse mediante Fetch desde MockAPI (no deben estar harcodeados en el HTML).
4. Maquetado UI: La interfaz debe ser responsiva y utilizar componentes de Bootstrap para la visualización de productos y el contenedor del carrito.

- **Estimación:** Representa el esfuerzo inicial de setup, diseño de la base de datos simulada y la integración de la primera capa de datos con la interfaz.

o **Esfuerzo Total Épica:** 8 (Escala Fibonacci).

Tarea 1: Configuración de estructura de directorios

- **Descripción:** Creación y ordenamiento de las carpetas locales del proyecto (/css, /js, /assets) para asegurar una arquitectura limpia.
- **Estimación:** Esfuerzo mínimo de organización interna.

o Esfuerzo: 1 | Complejidad: Baja | Incertidumbre: Baja

Tarea 2: Creación de index.html base con componentes comunes

- **Descripción:** Armado del esqueleto HTML e integración de las librerías de Bootstrap con Navbar y Footer funcionales.
- **Estimación:** Esfuerzo bajo enfocado en la estructura visual base.

o Esfuerzo: 1 | Complejidad: Baja | Incertidumbre: Baja

Tarea 3: Setup de recursos y datos iniciales en MockAPI

- **Descripción:** Configuración técnica de la plataforma externa y creación de las colecciones base (**users**, **products**, **orders**) con datos de prueba.
- **Estimación:** Requiere mapear correctamente los campos necesarios para las historias futuras.

o Esfuerzo: 2 | Complejidad: Baja | Incertidumbre: Baja

Tarea 4: Desarrollo de la lógica de Fetch para carga de catálogo

- **Descripción:** Implementación del script de JavaScript encargado de realizar la petición asíncrona (Fetch API) para obtener los productos de manera dinámica.
- **Estimación:** Implica manejo de asincronismo y control de errores en las peticiones.
- **Esfuerzo:** 2 | Complejidad: Media | Incertidumbre: Baja

Tarea 5: Maquetado de Cards de productos con Bootstrap

- **Descripción:** Diseño de la interfaz responsiva de la galería que recibe los datos de la API y los renderiza dinámicamente en pantalla.
- **Estimación:** Enfoque en estilos estructurados y visualización limpia de los datos inyectados.

o Esfuerzo: 2 | Complejidad: Baja | Incertidumbre: Baja

IX. Resultado del Sprint:

Funcionalidades implementadas:

- Estructura e Infraestructura Base: Inicialización exitosa de las tareas técnicas SCRUM-42, SCRUM-43, SCRUM-44, SCRUM-45 y SCRUM-46, completando el alcance definido para la primera parte de la base estructural.

Funcionalidades pendientes o replanificadas:

El desarrollo de la Base Estructural - Parte II (Lógica de Carrito y Checkout básico) permanece en el Backlog del proyecto para ser activado en el próximo periodo, manteniendo postergado el código de las historias de usuario principales (US-01, US-02, US-03) hasta contar con dichos cimientos.

X. Decisiones relevantes tomadas por el equipo:

Decisiones técnicas:

1. Postergación de lógica de almacenamiento local: Se decidió de manera unánime postergar la implementación de LocalStorage para sprints posteriores. Esto reduce la sobrecarga técnica actual del equipo y permite asegurar primero la estabilidad de las peticiones HTTP (Fetch) hacia la API.

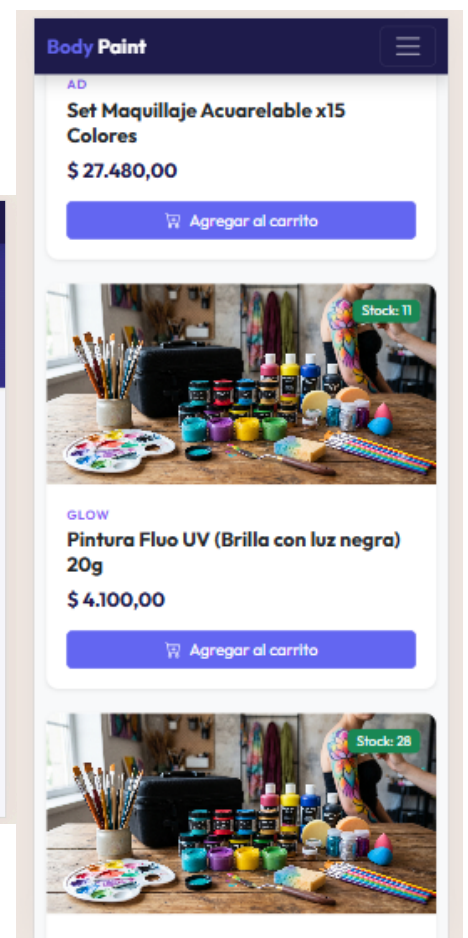
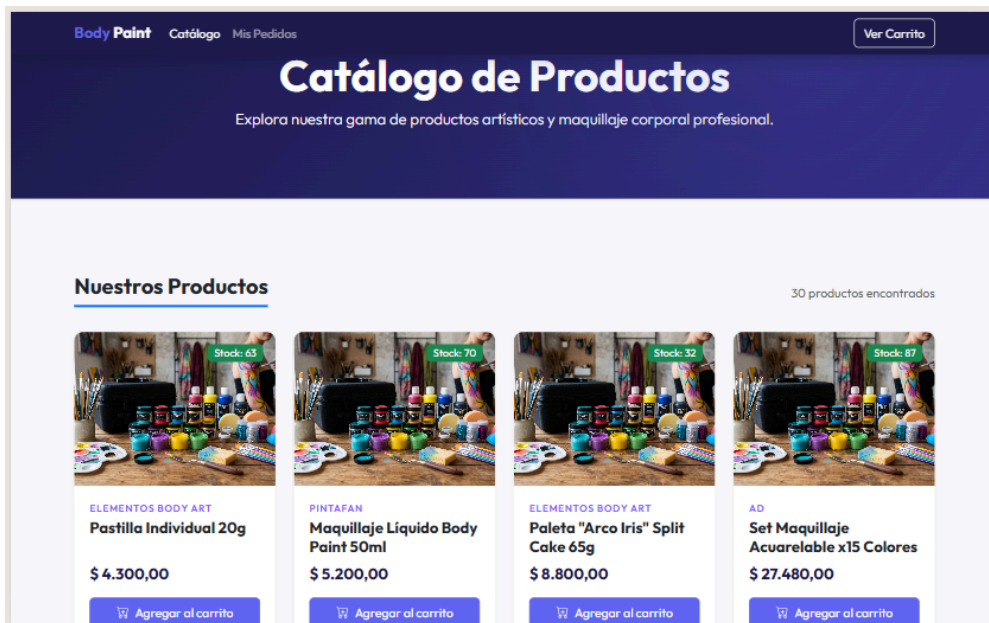
2. Estrategia de Desarrollo Paralelo (Reparto de Tareas): Para mitigar el impacto de la restricción horaria del equipo (estudio y trabajo), la Base Estructural se atomizó en las 5 tareas internas asignadas en paralelo, garantizando que el setup de MockAPI y el maquetado del frontend avanzaran de forma independiente y coordinada.

XI. Casos de Prueba:

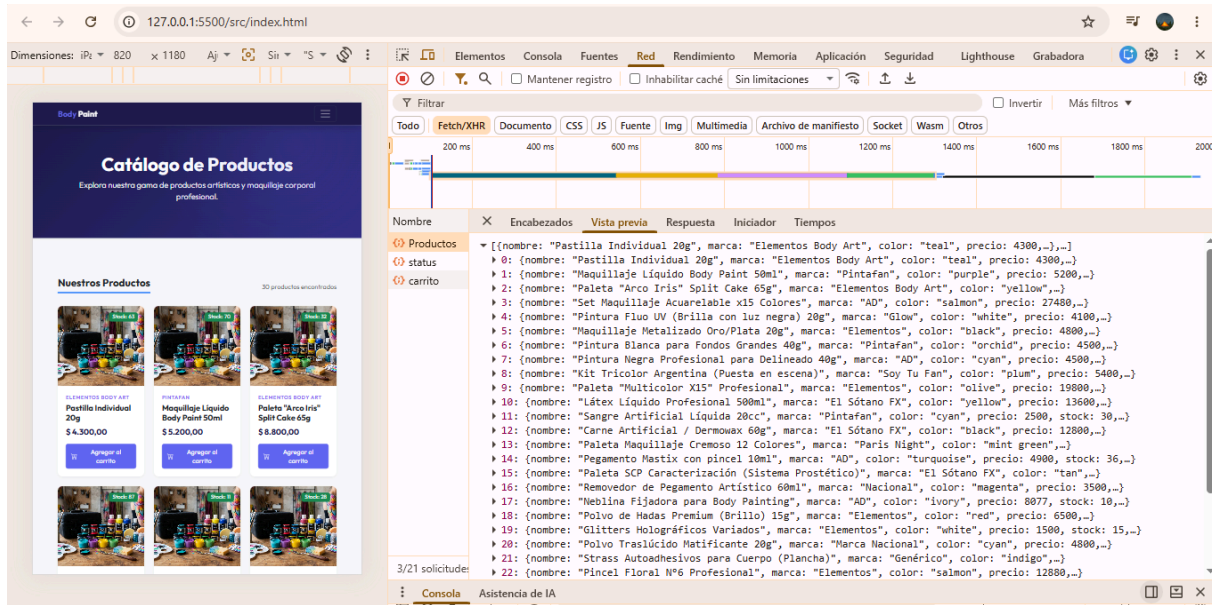
ID	Nombre del Caso	Precondiciones	Pasos a Ejecutar	Resultado Esperado	Estado
CP-01	Estructura Base y Plantilla	Entorno local configurado.	1. Iniciar servidor local. 2. Navegar a index.html.	Renderizado correcto de Navbar y Footer. Los archivos estáticos se cargan desde /css, /js y /assets.	✓

CP-02	Configuración MockAPI	Plataforma MockAPI accesible.	1. Ingresar al dashboard de MockAPI.	Existen las colecciones users, productos y pedidos con datos de prueba.	✓
CP-03	Carga Dinámica de Productos (Fetch)	index.html cargado, API activa.	1. Abrir página principal. 2. Inspeccionar red en DevTools.	Se realiza petición GET a MockAPI. Los productos se inyectan dinámicamente en el DOM (no hardcodeados).	✓
CP-04	Responsividad del Catálogo	index.html cargado.	1. Redimensionar ventana (Móvil, Tablet, Desktop).	Las Cards de productos se adaptan usando clases de Bootstrap, sin desbordamientos en pantalla.	✓

CP-01 y CP-04



CP-03



SPRINT 2

I. Objetivo del Sprint:

Implementación mínima de la lógica de selección de productos y generación de pedidos. El objetivo es permitir que exista un "Pedido" en el sistema para poder aplicarle cupones o reclamos más adelante.

II. Épicas y Tareas seleccionadas para el Sprint Backlog:

Épica: Desarrollo de la Base Estructural - Parte II: Lógica de Carrito y Checkout

Tarea 1: Lógica JS: Sumar productos a un array.

Tarea 2: Lógica JS: Cálculo del total.

Tarea 3: Interfaz: Botón de "Confirmar Compra".

Tarea 4: Integración API: Enviar Pedido (POST).

Tarea 5: Interfaz: Feedback de Pedido.

Tarea 6: Modelado JS: Crear clases base (Pedido, Cupon, Reclamo, Reenvio).

Tarea 7: Refactor Lógica JS: Adaptar el Carrito al nuevo modelo de Pedido.

III. Detalle de la Épica y sus Tareas Internas:

Épica: Desarrollo de la Base Estructural - Parte II: Lógica de Carrito y Checkout

- **Descripción:** Implementación mínima de la lógica de selección de productos y generación de pedidos. El objetivo es permitir que exista un "Pedido" en el sistema para poder aplicarle cupones o reclamos más adelante.

- **Criterios de Aceptación:**

1. El usuario puede elegir productos y ver el total.
2. Al tocar "Comprar", se envía la orden a MockAPI con un estado de "Pendiente".
3. Se genera un ID de pedido visible (necesario para los reclamos).

- **Estimación:** Representa el esfuerzo de desarrollo de la lógica de negocio del carrito, persistencia inicial en servidor y el diseño orientado a objetos para estructurar las entidades del dominio.

- o Esfuerzo: 5 (Escala Fibonacci).

Tarea 1: Lógica JS: Sumar productos a un array

- **Descripción:** Crear una variable (un array vacío) donde se guarden los objetos de los productos que el usuario elige.

- **Estimación:** Lógica básica de manipulación de estructuras de datos en memoria.

- o Esfuerzo: 1 | Complejidad: Baja | Incertidumbre: Baja

Tarea 2: Lógica JS: Cálculo del total

- **Descripción:** Una función simple que recorra ese array y sume los precios para mostrar el total en pantalla.

- **Estimación:** Operación matemática simple e iteración sobre la lista de elementos seleccionados.

- o Esfuerzo: 1 | Complejidad: Baja | Incertidumbre: Baja

Tarea 3: Interfaz: Botón de "Confirmar Compra"

- **Descripción:** Agregar un botón simple al final de la lista de productos seleccionados.

- **Estimación:** Maquetado básico de un elemento de acción en el contenedor del carrito.

- o Esfuerzo: 1 | Complejidad: Baja | Incertidumbre: Baja

Tarea 4: Integración API: Enviar Pedido (POST)

- **Descripción:** Programar que, al tocar "Confirmar", se mande ese array a la colección de pedidos de MockAPI.

- **Estimación:** Requiere gestionar peticiones asíncronas HTTP asumiendo la estructura remota establecida.

- o Esfuerzo: 2 | Complejidad: Media | Incertidumbre: Baja

Tarea 5: Interfaz: Feedback de Pedido

- **Descripción:** Mostrar un cartelito que diga "¡Pedido realizado con éxito! Tu número de pedido es [ID]".

- **Estimación:** Captura del identificador retornado por la API e inyección dinámica en la interfaz de usuario.

- o Esfuerzo: 1 | Complejidad: Baja | Incertidumbre: Baja

Tarea 6: Modelado JS: Crear clases base (Pedido, Cupon, Reclamo, Reenvio)

- **Descripción:** Diseño de las estructuras fundamentales del dominio para modelar los objetos del negocio con programación orientada a objetos.

- **Estimación:** Definición precisa de los atributos necesarios para el posterior desarrollo de las User Stories del cliente.

- o Esfuerzo: 2 | Complejidad: Media | Incertidumbre: Baja

Tarea 7: Refactor Lógica JS: Adaptar el Carrito al nuevo modelo de Pedido

- **Descripción:** Adecuación y reestructuración del código JavaScript existente del carrito para interactuar de forma nativa con los nuevos objetos de la clase Pedido.

- **Estimación:** Modificación técnica orientada a mantener la cohesión en la manipulación y persistencia de los datos.

- o Esfuerzo: 1 | Complejidad: Media | Incertidumbre: Baja

4. Resultado del Sprint:

Funcionalidades implementadas:

Lógica de Carrito y Checkout: Inicialización exitosa de las tareas técnicas SCRUM-57, SCRUM-58, SCRUM-59, SCRUM-60, SCRUM-61, SCRUM-62 y SCRUM-63, completando al 100% el alcance definido para la segunda parte de la base estructural.

Funcionalidades pendientes o replanificadas:

No quedan funcionalidades pendientes en este sprint. Al haberse completado la infraestructura de datos (Parte I) y la lógica de negocio del carrito (Parte II), el backlog

técnico estructural queda cerrado. El desarrollo de las User Stories principales (US-01, US-02, US-03) se encuentra listo para iniciar en el Sprint 3.

5. Decisiones relevantes tomadas por el equipo:

Decisiones técnicas:

1. Centralización de datos en API externa en lugar de persistencia local:

Manteniendo la estrategia previa de no delegar flujos complejos a LocalStorage, se priorizó la inyección de los ítems directamente en la estructura de datos del servidor. Esto garantiza que cada orden obtenga su respectivo identificador nativo desde MockAPI indispensable para implementar los flujos de negocio futuros de control de reclamos y cupones.

2. Estrategia de Desarrollo Paralelo (Reparto de Tareas): Para mitigar el impacto de la restricción horaria del equipo (estudio y trabajo), la Base Estructural se atomizó en las 7 tareas internas asignadas en paralelo, garantizando que el modelado lógico de las clases base y la programación de las llamadas asíncronas de la API avanzaran de forma independiente y coordinada.

6. Casos de Prueba:

ID	Nombre del Caso	Precondiciones	Pasos a Ejecutar	Resultado Esperado	Estado
CP-05	Agregar producto al array en memoria	Producto disponible en catálogo.	1. Clic en "Agregar al carrito".	El objeto del producto se añade al array del carrito en JS, sin persistir en LocalStorage.	✓
CP-06	Cálculo dinámico del total	Array de carrito con productos.	1. Agregar múltiples productos o modificar cantidades.	El sistema itera el array, suma los precios y actualiza el total en pantalla correctamente.	✓
CP-07	Envío de Pedido (POST a MockAPI)	Carrito con al menos un producto.	1. Clic en "Confirmar Compra".	Se envía una petición POST a la colección de pedidos en MockAPI. El pedido	✓

SPRINT 3

I. Objetivo del Sprint:

Implementar las funcionalidades de cara al cliente mediante el desarrollo de las User Stories principales, permitiendo la aplicación de cupones de descuento con validación de reglas de negocio, la gestión de reclamos dentro de los plazos establecidos y la tramitación de solicitudes de reenvío de productos con validación de stock.

II. Historias de Usuario seleccionadas para el Sprint Backlog:

US-01: Cupón de Descuento

Tarea 1: [Frontend] Maquetado del campo de cupones.

Tarea 2: [Backend/MockAPI] Definición del recurso "Cupones".

Tarea 3: [Lógica JS] Validación de reglas de negocio.

Tarea 4: [Integración] Persistencia de uso.

Tarea 5: [Arquitectura] Refactor de la Pantalla del Carrito.

US-02: Gestión de Reclamos

Tarea 1: [Interfaz] Botón de Reclamo en Historial.

Tarea 2: [Lógica JS] Función de Validación de Fecha.

Tarea 3: [Interfaz] Formulario de Reclamo.

Tarea 4: [Integración API] POST de Reclamo.

Tarea 5: [Lógica de Cierre] Cambio de Estado Automático.

US-03: Solicitud de Reenvío

Tarea 1: Interfaz de selección.

Tarea 2: Lógica de validación de stock.

Tarea 3: Persistencia de Reenvío.

III. Detalle de las User Stories y sus Tareas Internas:

US-01: Cupón de Descuento

- **Estado actual:** Implementación y codificación de la funcionalidad analizada en el Sprint 0, ejecutando el desarrollo técnico sobre la base del sistema.

- **Descripción:** Como cliente, quiero ingresar un código de descuento en el carrito de compras para obtener una reducción en el precio final de mi pedido.

- **Criterios de Aceptación:**

1. El sistema debe validar que el cupón exista en la base de datos (MockAPI).
2. El descuento solo se aplica si el monto total es superior al valor del cupón.
3. Al confirmar la compra, el cupón debe quedar invalidado para ese usuario (o marcado como usado).

- **Estimación:** Requiere validación de reglas de negocio y actualización de montos en tiempo real.

o Esfuerzo: 5 (Escala Fibonacci) | Complejidad: Media | Incertidumbre: Baja

Tarea 1: [Frontend] Maquetado del campo de cupones

- **Descripción:** Crear el input y el botón de "Aplicar" en la interfaz del carrito usando Bootstrap.

- **Estimación:** Maquetado simple sobre la estructura visual existente.

o Esfuerzo: 1 | Complejidad: Baja | Incertidumbre: Baja

Tarea 2: [Backend/MockAPI] Definición del recurso "Cupones"

- **Descripción:** Crear una colección en MockAPI que contenga: código (ej: "PROMO10"), valor (monto del descuento) y estado (activo/usado).

- **Estimación:** Modelado de datos directo en la API externa.

o Esfuerzo: 1 | Complejidad: Baja | Incertidumbre: Baja

Tarea 3: [Lógica JS] Validación de reglas de negocio

- **Descripción:** Programar la función en JavaScript que haga el fetch a MockAPI, verifique si el código es válido y reste el monto al total del carrito.

- **Estimación:** Manejo de asincronismo y lógica de cálculo del nuevo total condicionado.

o Esfuerzo: 1 | Complejidad: Media | Incertidumbre: Baja

Tarea 4: [Integración] Persistencia de uso

- **Descripción:** Lógica para enviar un PUT o POST a la API al finalizar el pedido, asegurando que el cupón no se pueda volver a usar.
- **Estimación:** Actualización del estado del recurso en el servidor al confirmar la transacción.

o Esfuerzo: 1 | Complejidad: Baja | Incertidumbre: Baja

Tarea 5: [Arquitectura] Refactor de la Pantalla del Carrito

- **Descripción:** Adecuación estructural de la vista del carrito para soportar la re-renderización de los montos tras aplicar el beneficio.
- **Estimación:** Modificación técnica para asegurar la consistencia visual y de datos.

o Esfuerzo: 1 | Complejidad: Media | Incertidumbre: Baja

US-02: Gestión de Reclamos

- **Estado actual:** Construcción y despliegue del módulo de post-venta, integrando los controles de fechas con la persistencia en el servidor.
- **Descripción:** Como cliente, quiero realizar reclamos dentro de los 5 días posteriores a la entrega para solucionar problemas.

- **Criterios de Aceptación:**

1. El sistema debe habilitar la opción de reclamo solo hasta 5 días hábiles después de la entrega.
2. El usuario debe poder ingresar el motivo del reclamo mediante un campo de texto.
3. Si no hay reclamo en ese plazo, el pedido se marca como finalizado automáticamente.

- **Estimación:** Esfuerzo bajo ya que se reutilizará la lógica de validación de fechas de proyectos anteriores, aunque con incertidumbre media por la integración de feriados.

o Esfuerzo: 3 (Escala Fibonacci) | Complejidad: Baja | Incertidumbre: Media (por el cálculo de días hábiles)

Tarea 1: [Interfaz] Botón de Reclamo en Historial

- **Descripción:** Agregar el botón "Realizar Reclamo" en la vista de pedidos del cliente, aplicando la lógica de visibilidad (solo si el estado es 'Entregado').

- **Estimación:** Renderizado condicional basado en el estado del pedido provisto por la API.

o Esfuerzo: 0.5 | Complejidad: Baja | Incertidumbre: Baja

Tarea 2: [Lógica JS] Función de Validación de Fecha

- **Descripción:** Desarrollar el script que compare la fecha_entrega de MockAPI con la fecha actual y verifique si está dentro del rango de los 5 días hábiles.

- **Estimación:** Lógica de fechas contemplando la restricción de días laborables.

o Esfuerzo: 1 | Complejidad: Media | Incertidumbre: Media

Tarea 3: [Interfaz] Formulario de Reclamo

- **Descripción:** Crear una ventana modal o una vista simple con un campo de texto (textarea) para que el cliente describa el problema.

- **Estimación:** Implementación de componente interactivo para la captura de texto de manera responsiva.

o Esfuerzo: 0.5 | Complejidad: Baja | Incertidumbre: Baja

Tarea 4: [Integración API] POST de Reclamo

- **Descripción:** Programar el envío de los datos (ID de pedido, motivo y fecha) a una nueva colección en MockAPI llamada reclamos.

- **Estimación:** Alta de un nuevo registro vinculado formalmente al identificador del pedido de origen.

o Esfuerzo: 0.5 | Complejidad: Baja | Incertidumbre: Baja

Tarea 5: [Lógica de Cierre] Cambio de Estado Automático

- **Descripción:** Implementar la función que, al detectar que pasaron los 5 días sin reclamos, realice un PUT a MockAPI para cambiar el estado del pedido original a "Finalizado".

- **Estimación:** Lógica de control para la actualización automatizada o reactiva del estado en el servidor.

o Esfuerzo: 0.5 | Complejidad: Baja | Incertidumbre: Baja

US-03 - Solicitud de Reenvío

- **Estado actual:** Programación de la lógica logística y de inventario, resolviendo la complejidad técnica de la verificación de stock planteada en el inicio.

- **Descripción:** Como cliente, quiero solicitar el reenvío de productos (por faltantes o defectos) para recibir lo que corresponde de mi pedido original.

- **Criterios de Aceptación:**

1. El cliente debe poder seleccionar si el reenvío es por la totalidad o solo por algunos ítems del pedido.
2. El sistema debe validar que el pedido original esté en estado "Entregado" y que exista un reclamo previo aprobado o en análisis.
3. La solicitud debe generar un nuevo registro en el recurso de "Reenvíos" de MockAPI vinculado al ID del pedido original.
4. Regla de Stock: El sistema debe notificar si no hay stock disponible para el reenvío inmediato.

- **Estimación:** Se considera de esfuerzo alto no solo por la lógica de selección de productos, sino por la incertidumbre crítica respecto a la disponibilidad de stock. Si el producto a reenviar no está disponible, el sistema debe contemplar un flujo alternativo, lo que aumenta el riesgo técnico y de negocio.

o Esfuerzo: 8 (Escala Fibonacci) | Complejidad: Alta (implica lógica de logística y nuevos estados de pedido) | Incertidumbre: Alta

Tarea 1: Interfaz de selección

- **Descripción:** Crear la vista para marcar qué productos específicos del pedido necesitan reenvío.

- **Estimación:** Interfaz con selectores múltiples mapeando el detalle de ítems del pedido original.

o Esfuerzo: 2 | Complejidad: Media | Incertidumbre: Baja

Tarea 2: Lógica de validación de stock

- **Descripción:** Programar el chequeo contra la colección de productos en MockAPI.

- **Estimación:** Cruce asíncrono de datos entre las cantidades solicitadas y las existencias reales en el servidor.

o Esfuerzo: 3 | Complejidad: Alta | Incertidumbre: Alta

Tarea 3: Persistencia de Reenvío

- **Descripción:** Configurar el recurso en la API para guardar estas solicitudes especiales.
- **Estimación:** Creación de la colección y persistencia de las estructuras de reenvío vinculadas.

o Esfuerzo: 3 | Complejidad: Media | Incertidumbre: Baja

IV. Resultado del Sprint:

Funcionalidades implementadas:

Funcionalidades del Cliente: Finalización exitosa de las tareas de desarrollo, maquetado e integración correspondientes a las historias de usuario de Cupón de Descuento (US-01), Gestión de Reclamos (US-02) y Solicitud de Reenvío (US-03), logrando la conectividad total con los nuevos recursos configurados en MockAPI.

Funcionalidades pendientes o replanificadas:

No quedan funcionalidades pendientes en este sprint. Al haberse completado el desarrollo del código fuente y las reglas de negocio de las tres User Stories principales de este bloque, el núcleo funcional del MVP se encuentra cerrado y listo para las fases finales de integración y pruebas del sistema.

V. Decisiones relevantes tomadas por el equipo:

Decisiones técnicas:

1. Validación relacional estricta en el servidor: Para cumplir con las reglas de negocio de cupones, reclamos y reenvíos, se optó por un esquema de validación lógica previo a cada actualización en MockAPI. Esto asegura que no se procesen solicitudes de reenvío sin estados "Entregado" ni cupones sin saldos mínimos, resguardando la integridad de la base de datos sin depender puramente del estado local del cliente.

2. Estrategia de Desarrollo Paralelo (Reparto de Tareas): Con el fin de maximizar el rendimiento frente a las limitaciones de tiempo, el equipo dividió el desarrollo de las subtarefas de interfaz (maquetados de cupones y formularios modales de reclamo) en paralelo con la programación de las funciones asíncronas de la API (peticiones POST/PUT y chequeos de stock), garantizando la entrega simultánea de las tres historias de usuario al cierre del sprint.

VI. Casos de Prueba:

US-01: Cupón de Descuento

ID	Nombre del Caso	Precondiciones	Pasos a Ejecutar	Resultado Esperado	Estado
CP-10	Aplicar cupón válido y activo	Carrito con productos (monto > cupón), cupón en MockAPI (estado: activo).	1. Ingresar código. 2. Clic en "Aplicar".	Descuento aplicado al total, mensaje de éxito visible.	✓
CP-11	Bloqueo de cupón ya utilizado	Cupón en MockAPI con estado: usado.	1. Ingresar código. 2. Clic en "Aplicar".	Total no cambia, sistema muestra error: "Este cupón ya ha sido utilizado".	✓
CP-12	Tope de descuento al subtotal	Carrito con productos cuyo total es inferior al valor del cupón.	1. Ingresar código válido. 2. Clic en "Aplicar".	Descuento se ajusta al valor del subtotal. Total a pagar es \$0,00 (sin saldo negativo).	✓
CP-13	Invalidación post-compra	Cupón válido aplicado en el carrito.	1. Click en "Confirmar Compra".	Se envía PUT/POST a MockAPI marcando el cupón como usado.	✓

CP-10

The screenshot displays a web browser window with the URL `127.0.0.1:5500/src/carrito.html`. The page title is "Body Paint" and it includes navigation links for "Catálogo" and "Mis Pedidos". A "Seguir comprando" button is visible in the top right.

The main content area is titled "Tu Carrito de Compras" and contains a table with the following items:

Producto	Precio	Cantidad	Subtotal
Maquillaje Líquido Body Paint 50ml	\$ 5.200,00	1	\$ 5.200,00
Pastilla Individual 20g	\$ 4.300,00	1	\$ 4.300,00

To the right of the cart items is a "Resumen del Pedido" (Order Summary) box. It includes a "Código de descuento" field with the value "BIENVENIDA10" and an "Aplicar" button. Below this, it shows the "Subtotal: \$ 9.500,00", "Descuento: -\$ 950,00", and a final "Total: \$ 8.550,00". A green "Confirmar Compra" button is at the bottom.

On the right side of the browser window, a developer tool is open, showing the "Elements" panel. It displays a list of elements with their names, status, type, initiator, and size. The elements listed are:

- carrito (status: 304, type: fetch, initiator: carrito.js:5, size: 0,5 kB)
- status (status: 304, type: fetch, initiator: fiveserver.js:1, size: 0,3 kB)
- cupones (status: 200, type: fetch, initiator: pedidos.js:31, size: 9,4 kB)

CP-11

127.0.0.1:5500/src/carrito.html

Dimensiones: Nest Hub Max 1280 x 800 75% Sin limitaciones "Save-Data": predeterminado

Body Paint Catálogo Mis Pedidos Seguir comprando

Tu Carrito de Compras

Producto	Precio	Cantidad	Subtotal
Maquillaje Líquido Body Paint 50ml	\$ 5.200,00	1	\$ 5.200,00

Resumen del Pedido

Código de descuento
super20
Aplicar
Cupón inválido o ya utilizado.

Subtotal: \$ 5.200,00

Total: \$ 5.200,00

Procesando...

Consola

Fetch/XHR

Documento

CSS

JS

Fuente

Img

Multimedia

Archivo de manifiesto

Socket

Wasm

Otros

100.000 ms

200.000 ms

300.000 ms

400.000 ms

500.000 ms

Nombre

carrito

status

cupones

pedidos

3

2

2

carrito

1

cupones

[{"codigo_cupon": "BIENVENIDA10-3", "client_id": 36, "codigo": "BIENVENIDA10", "codigo_cupon": "BIENVENIDA10-3", "estado": "inactivo", "id": "3", "monto_descuento": "", "productos_abarcados": "Todos", "tipo": "porcentaje", "usado": true, "validez": "2027-06-08T14:27:27.000Z", "valor": 10}]]

CP-12

127.0.0.1:5500/src/carrito.html

Dimensiones: Nest Hub Max 1280 x 800 75% Sin limitaciones "Save-Data": predeterminado

Body Paint Catálogo Mis Pedidos Seguir comprando

Tu Carrito de Compras

Producto	Precio	Cantidad	Subtotal
Stencil / Plantilla para Body Art	\$ 1.500,00	1	\$ 1.500,00

Resumen del Pedido

Código de descuento
DESC2000
Aplicar
¡Cupón aplicado correctamente!

Subtotal: \$ 1.500,00

Descuento: - \$ 1.500,00

Total: \$ 0,00

Confirmar Compra

Consola

Fetch/XHR

Documento

CSS

JS

Fuente

Img

Multimedia

Archivo de manifiesto

Socket

Wasm

Otros

50.000 ms

100.000 ms

150.000 ms

200.000 ms

250.000 ms

Nombre

carrito

status

cupones

Estado

304

304

200

Tipo

fetch

fetch

fetch

Iniciador

carrito.js:5

firebase.js:1

pedidos.js:31

Tama...

0,3 kB

0,3 kB

9,7 kB

Tiem...

186 ms

9 ms

950 ms

CP-13

127.0.0.1:5500/src/carrito.html

Dimensiones: Sa 412 x 915

Body Paint

Tu Carrito de Compras

Maquillaje

¡Compra Exitosa!

¡Pedido realizado con éxito!

Tu número de pedido es: 25

Podés hacer el seguimiento o iniciar un reclamo desde Mis Pedidos

Aceptar

Procesando...

Consola

Fetch/XHR

Documento

CSS

JS

Fuente

Img

Multimedia

Archivo de manifiesto

Socket

Wasm

Otros

5000 ms

10.000 ms

15.000 ms

20.000 ms

25.000 ms

30.000 ms

35.000 ms

40.000 ms

45.000 ms

50.000 ms

55.000 ms

60.000 ms

65.000 ms

70.000 ms

Nombre

carrito

status

cupones

pedidos

3

2

2

carrito

1

URL De Solicitud

Método De La Solicitud

Código De Estado

Dirección Remota

Política De Referencia

https://6a1dcdbfbcc4f20d5ca522b1.mockapio.io/tpi/cupones/3

PUT

200 OK

99.83.220.108:443

strict-origin-when-cross-origin

Encabezados de respuesta

Access-Control-Allow-Headers

Access-Control-Allow-Methods

Access-Control-Allow-Origin

Content-Length

Content-Type

Date

Nel

Report-To

Reporting-Endpoints

X-Requested-With,Content-Type,Cache-Control,access_token

GET,PUT,POST,DELETE,OPTIONS

*

233

application/json

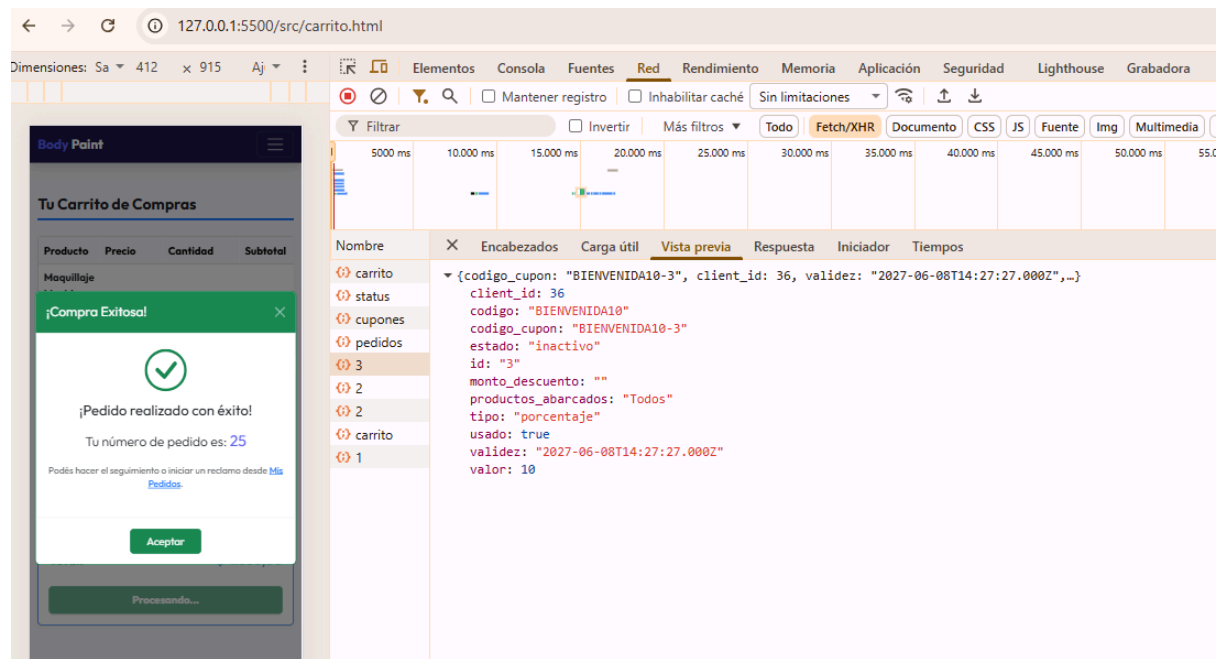
Sat, 13 Jun 2026 18:14:40 GMT

[{"report_to":"heroku-nel","response_headers":["Via"],"max_age":3600,"success_fraction":0.01,"failure_fraction":0.1}]

[{"group":"heroku-nel","endpoints":[{"url":"https://nel.heroku.com/reports?ts=Sj4E6UdRXU1kUPUzA1Gvik2KUnGgYceffAzmSfripA%3D%u0026sid=1b10b0ff-8a76-4548-befa-353fc6c6c045%u0026ts=1781374480"}],"max_age":3600}]

heroku-nel="https://nel.heroku.com/reports?ts=Sj4E6UdRXU1kUPUzA1Gvik2KUnGgYceffAzmSfripA%3D%u0026sid=1b10b0ff-8a76-4548-befa-353fc6c6c045&ts=1781374480"

Heroku



US-02: Gestión de Reclamos

ID	Nombre del Caso	Precondiciones	Pasos a Ejecutar	Resultado Esperado	Estado
CP-14	Habilitación de Reclamo	Pedido en estado "Entregado", fecha de entrega < 5 días hábiles.	1. Ingresar al historial de pedidos.	Botón "Realizar Reclamo" es visible y clickeable.	✓
CP-15	Envío de Reclamo exitoso	Pedido "Entregado" en plazo, formulario modal abierto.	1. Ingresar motivo en el campo de texto. 2. Clic en Enviar.	POST a colección /reclamos en MockAPI vinculado al ID de pedido.	✓
CP-16	Cierre automático de pedido	Pedido "Entregado", fecha de entrega > 5 días hábiles.	1. Ingresar al historial de pedidos.	Botón oculto. Sistema realiza PUT cambiando estado del pedido a "Finalizado".	✓

US-03 - Solicitud de Reenvío

ID	Nombre del Caso	Precondiciones	Pasos a Ejecutar	Resultado Esperado	Estado
CP-17	Reenvío exitoso con validación	Pedido "Entregado", reclamo previo activo. Productos con stock.	1. Seleccionar ítems a reenviar. 2. Confirmar solicitud.	POST a /reenvios exitoso. Validación asíncrona de stock superada.	✓
CP-18	Bloqueo por falta de stock	Productos seleccionados no tienen stock suficiente en MockAPI.	1. Confirmar solicitud de reenvío.	Operación rechazada. Sistema notifica: "No hay stock disponible para el reenvío inmediato".	✓
CP-19	Intento de reenvío inválido	Pedido sin reclamo previo asociado o no entregado.	1. Intentar acceder a interfaz de reenvío.	Interfaz bloqueada o validación estricta rechaza la petición.	✓